i2i Systems

Telecommunication Database Management Project

Developer: Ramazan BIYIK

Date: 21.04.2026

GitHub Repository:
https://github.com/Ramazanbiyiik/telco-project

Project Overview

This project focuses on building a robust database foundation and performing data analysis for a telecommunications system, developed as part of an assignment for i2i Systems. The primary objective was to set up a fully containerized Oracle XE environment, design a relational schema for telecom operations, and import customer and tariff datasets. Beyond basic SQL querying, the project emphasizes database architecture best practices, including automated deployment, data integrity, and performance optimization through strategic indexing. The final phase involves extracting meaningful business insights from the data, such as customer distribution, package exhaustion rates, and payment habits.

Database Schema

The database schema is strictly designed to accommodate and accurately reflect the provided telecom dataset. It consists of three core tables: TARIFFS, CUSTOMERS, and MONTHLY_STATS. Primary Key and Foreign Key constraints are applied across all tables to maintain referential integrity.

To optimize query performance, specific indexing strategies were applied:

- **B-Tree Indexes:** Applied to frequently searched and joined columns such as CITY, SIGNUP_DATE, and CUSTOMER_ID.
- **Bitmap Index:** Created on the PAYMENT_STATUS column in the MONTHLY_STATS table to accelerate analytical queries, as this column contains low-cardinality data.

Setup & Infrastructure

To ensure the project is easily reproducible, the database infrastructure is containerized using Docker. A custom docker-compose.yml file is configured to spin up an Oracle 21c XE instance.

An automated database seeding mechanism was implemented for the initial setup. By mapping the initialization script (TABLE_CREATION_SCRIPTS.sql) to the container's /container-entrypoint-initdb.d directory, the Docker setup automatically handles the following:

- Switches the session to the correct pluggable database (XEPDB1).
- Routes the operations to the dedicated application schema (telco_dev).
- Executes the DDL scripts to build the tables and indexes.

```
C:\Users\ramaz\OneDrive\Desktop\telco-project>docker-compose up -d
time="2026-04-20T14:03:59+03:00" level=warning msg="C:\\Users\\ramaz\\OneDrive\\Desktop\\telco-project\\docker-compose.yml: the attribute `vers
ion` is obsolete, it will be ignored, please remove it to avoid potential confusion"
[+] up 3/3
 ✓ Network telco-project_default    Created                                                                                    0.0s
 ✓ Volume telco-project_oracle-data Created                                                                                    0.0s
 ✓ Container i2i_oracle_db          Started                                                                                    0.3s
```

1. Tariff-Based Customer Queries

1.1 List the customers who are subscribed to the 'Kobiye Destek' tariff.

```
SELECT c.CUSTOMER_ID, c.NAME AS CUSTOMER_NAME, t.NAME AS TARIFF_NAME

FROM CUSTOMERS c

JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID

WHERE t.NAME = 'Kobiye Destek';
```

Results 1

SELECT c.CUSTOMER_ID, c.NAME AS CUSTOM

Enter a SQL expression to filter results (use Ctrl+Space)

Ctrl+click to open SQL console

		CUSTOMER_NAME	TARIFF_NAME
1	8,970	Ahmet	Kobiye Destek
2	8,975	Tuğba	Kobiye Destek
3	8,979	Kadir	Kobiye Destek
4	8,981	Zehra	Kobiye Destek
5	8,987	İsmail	Kobiye Destek
6	8,999	Yasin	Kobiye Destek
7	9,000	Adem	Kobiye Destek
8	9,005	Fatih	Kobiye Destek
9	9,007	Furkan	Kobiye Destek
10	9,013	Zehra	Kobiye Destek
11	9,015	Serkan	Kobiye Destek
12	9,017	Hakan	Kobiye Destek
13	9,018	Songül	Kobiye Destek
14	9,022	Havva	Kobiye Destek
15	9,023	Ömer	Kobiye Destek
16	9,025	Zeynep	Kobiye Destek
17	9,029	Kübra	Kobiye Destek

Refresh Save Cancel Export data 200 200+

1.2 Find the newest customer who subscribed to this tariff.

```
SELECT c.CUSTOMER_ID, c.NAME, c.SIGNUP_DATE

FROM CUSTOMERS c

JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID

WHERE t.NAME = 'Kobiye Destek'

ORDER BY c.SIGNUP_DATE DESC

FETCH FIRST 1 ROWS ONLY;
```

Results 1

SELECT c.CUSTOMER_ID, c.NAME, c.SIGNUP_D

	CUSTOMER_ID	NAME	SIGNUP_DATE
1	8,798	Hatice	2028-07-03 00:00:00.000

2. Tariff Distribution

2.1 Find the distribution of tariffs among the customers.

```
SELECT t.NAME AS TARIFF_NAME, COUNT(c.CUSTOMER_ID) AS TOTAL_CUSTOMERS

FROM TARIFFS t

LEFT JOIN CUSTOMERS c ON t.TARIFF_ID = c.TARIFF_ID

GROUP BY t.NAME

ORDER BY TOTAL_CUSTOMERS DESC;
```

Results 1

SELECT t.NAME AS TARIFF_NAME, COUNT(c.C

	TARIFF_NAME	TOTAL_CUSTOMERS
1	Kurumsal SMS	2,577
2	Genç Dinamik	2,527
3	Kobiye Destek	2,483
4	Çalışan GB	2,413

3.1 Identify the earliest customers to sign up.

```
SELECT CUSTOMER_ID, NAME, SIGNUP_DATE, CITY

FROM CUSTOMERS

WHERE SIGNUP_DATE = (SELECT MIN(SIGNUP_DATE) FROM CUSTOMERS);
```

Results 1

SELECT CUSTOMER_ID, NAME, SIGNUP_DATE,

	CUSTOMER_ID	NAME	SIGNUP_DATE	CITY
1	9,021	Kübra	2025-01-05 00:00:00.000	GİRESUN
2	9,548	Havva	2025-01-05 00:00:00.000	BARTIN
3	9,298	Serkan	2025-01-05 00:00:00.000	BAYBURT
4	8,196	Havva	2025-01-05 00:00:00.000	BİNGÖL
5	7,992	Furkan	2025-01-05 00:00:00.000	ŞIRNAK
6	7,606	Burak	2025-01-05 00:00:00.000	YOZGAT
7	7,336	Zehra	2025-01-05 00:00:00.000	KAHRAMANMAF
8	7,851	Hacer	2025-01-05 00:00:00.000	ARDAHAN
9	8,877	Furkan	2025-01-05 00:00:00.000	EDİRNE
10	8,765	Dilek	2025-01-05 00:00:00.000	BATMAN
11	6,409	Emre	2025-01-05 00:00:00.000	DÜZCE
12	5,109	Mustafa	2025-01-05 00:00:00.000	AĞRI
13	4,889	Mehmet	2025-01-05 00:00:00.000	KOCAELİ
14	4,674	Dilek	2025-01-05 00:00:00.000	ARTVİN
15	4,848	Osman	2025-01-05 00:00:00.000	ŞIRNAK
16	5,712	Rabia	2025-01-05 00:00:00.000	BATMAN
17	5,774	Kübra	2025-01-05 00:00:00.000	HAKKARİ

3.2 Find the distribution of these earliest customers across different cities.

```
SELECT CITY, COUNT(*) AS CUSTOMER_COUNT
FROM CUSTOMERS
WHERE SIGNUP_DATE = (SELECT MIN(SIGNUP_DATE) FROM CUSTOMERS)
GROUP BY CITY;
```

Results 1

SELECT CITY, COUNT(*) AS CUSTOMER_COUN

	CITY	CUSTOMER_COUNT
1	GİRESUN	1
2	BARTIN	1
3	BAYBURT	1
4	BİNGÖL	1
5	ŞIRNAK	2
6	YOZGAT	1
7	KAHRAMANMAF	1
8	ARDAHAN	1
9	EDİRNE	1
10	BATMAN	2
11	DÜZCE	1
12	AĞRI	1
13	KOCAELİ	1
14	ARTVİN	1

4. Missing Monthly Records

4.1 Identify the IDs of these missing customers.

```
SELECT c.CUSTOMER_ID, c.NAME
FROM CUSTOMERS c
LEFT JOIN MONTHLY_STATS m ON c.CUSTOMER_ID = m.CUSTOMER_ID
WHERE m.ID IS NULL;
```

Results 1

SELECT c.CUSTOMER_ID, c.NAME FROM CUST

	CUSTOMER_ID	NAME
1	953	Ali
2	988	Esra
3	458	Metin
4	463	Osman
5	467	Hasan
6	205	Mahmut
7	211	Hasan
8	218	Merve
9	221	Merve
10	229	Elif
11	233	Halil
12	301	Ayşe
13	6	Fadime
14	10	Hakan
15	31	Serkan

4.2 Find the distribution of these missing customers across different cities.

```
SELECT c.CITY, COUNT(*) AS MISSING_RECORD_COUNT

FROM CUSTOMERS c

LEFT JOIN MONTHLY_STATS m ON c.CUSTOMER_ID = m.CUSTOMER_ID

WHERE m.ID IS NULL

GROUP BY c.CITY;
```

Results 1

SELECT c.CITY, COUNT(*) AS MISSING_RECOR

	CITY	MISSING_RECORD_COUNT
1	İZMİR	2
2	SAMSUN	1
3	KONYA	1
4	ERZURUM	1
5	ESKİŞEHİR	1
6	OSMANİYE	3
7	ÇANKIRI	1
8	ŞIRNAK	1
9	KARABÜK	1
10	TEKİRDAĞ	1
11	BİTLİS	2
12	MUŞ	2
13	KIRŞEHİR	1
14	GAZİANTEP	1
15	SİİRT	1

5. Usage Analysis

5.1 Find the customers who have used at least 75% of their data limit.

```
SELECT c.CUSTOMER_ID, c.NAME, m.DATA_USAGE, t.DATA_LIMIT

FROM CUSTOMERS c

JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID

JOIN MONTHLY_STATS m ON c.CUSTOMER_ID = m.CUSTOMER_ID

WHERE m.DATA_USAGE >= (t.DATA_LIMIT * 0.75);
```

Results 1

SELECT c.CUSTOMER_ID, c.NAME, m.DATA_US

	CUSTOMER_ID	NAME	DATA_USAGE	DATA_LIMIT
1	8,974	Emine	9,727.16	10,240
2	9,006	Ömer	9,254.38	10,240
3	9,052	Ali	9,461.14	10,240
4	9,067	Burak	9,762.84	10,240
5	9,071	Enes	9,091.15	10,240
6	9,078	Havva	9,459.83	10,240
7	8,779	Elif	8,852.42	10,240
8	8,784	Aysel	9,208.39	10,240
9	8,788	Ayşe	8,865.96	10,240
10	8,792	Kadir	9,579.71	10,240
11	8,803	Metin	8,730.88	10,240
12	8,824	Ayşe	9,134.98	10,240
13	8,860	Hasan	7,890.83	10,240
14	9,717	Merve	8,493.24	10,240
15	9,721	Meryem	8,920.16	10,240
16	9,727	Esra	9,128.5	10,240
17	9,798	Ramazan	9,141.59	10,240

5.2 Identify the customers who have completely exhausted all of their package limits.

```
SELECT c.CUSTOMER_ID, c.NAME

FROM CUSTOMERS c

JOIN TARIFFS t ON c.TARIFF_ID = t.TARIFF_ID

JOIN MONTHLY_STATS m ON c.CUSTOMER_ID = m.CUSTOMER_ID

WHERE m.DATA_USAGE >= t.DATA_LIMIT

  AND m.MINUTE_USAGE >= t.MINUTE_LIMIT

  AND m.SMS_USAGE >= t.SMS_LIMIT;
```

Results 1

SELECT c.CUSTOMER_ID, c.NAME FROM CUST

CUSTOMER_ID	NAME

6. Payment Analysis

6.1 Find the customers who have unpaid fees.

```
SELECT c.CUSTOMER_ID, c.NAME, m.PAYMENT_STATUS

FROM CUSTOMERS c

JOIN MONTHLY_STATS m ON c.CUSTOMER_ID = m.CUSTOMER_ID

WHERE m.PAYMENT_STATUS <> 'PAID';
```

SELECT c.CUSTOMER_ID, c.NAME, m.PAYMEN

	CUSTOMER_ID	NAME	PAYMENT_STATUS
1	8,970	Ahmet	UNPAID
2	8,971	Songül	UNPAID
3	8,973	Kübra	LATE
4	8,975	Tuğba	UNPAID
5	8,976	Özlem	UNPAID
6	8,977	Burak	UNPAID
7	8,979	Kadir	LATE
8	8,986	Ömer	UNPAID
9	8,987	İsmail	LATE
10	8,990	Enes	LATE
11	9,004	Sevim	LATE
12	9,006	Ömer	UNPAID
13	9,014	Enes	LATE
14	9,015	Serkan	LATE
15	9,019	Hasan	LATE
16	9,020	İbrahim	LATE

6.2 Find the distribution of all payment statuses across the different tariffs.

```
SELECT t.NAME AS TARIFF_NAME, m.PAYMENT_STATUS, COUNT(*) AS STATUS_COUNT

FROM TARIFFS t

JOIN CUSTOMERS c ON t.TARIFF_ID = c.TARIFF_ID

JOIN MONTHLY_STATS m ON c.CUSTOMER_ID = m.CUSTOMER_ID

GROUP BY t.NAME, m.PAYMENT_STATUS

ORDER BY t.NAME, m.PAYMENT_STATUS;
```

Results 1

SELECT t.NAME AS TARIFF_NAME, m.PAYMEN

	TARIFF_NAME	PAYMENT_STATUS	STATUS_COUNT
1	Genç Dinamik	LATE	372
2	Genç Dinamik	PAID	1,792
3	Genç Dinamik	UNPAID	352
4	Kobiye Destek	LATE	392
5	Kobiye Destek	PAID	1,719
6	Kobiye Destek	UNPAID	360
7	Kurumsal SMS	LATE	368
8	Kurumsal SMS	PAID	1,796
9	Kurumsal SMS	UNPAID	403
10	Çalışan GB	LATE	365
11	Çalışan GB	PAID	1,692
12	Çalışan GB	UNPAID	339